

Multi-Agent Failure Modes & Mitigations

Bound the loop, watch the cost, and catch drift before it reaches production

Source: <https://learn.microsoft.com/en-us/agent-framework/agents/looping>

Infinite loops

- Framework's own warning, verbatim: `"Always bound autonomous loops. A completion condition can fail, a model can stall, and an evaluator can be probabilistic."`
- `AgentLoopMiddleware(predicate, max_iterations=N)` bounds a single agent re-invoking itself until a completion condition is met (default max: 10 runs)
- `AgentLoopMiddleware.with_judge(judge_client, criteria=[...], max_iterations=N)` is a judge-driven variant — structurally the closest built-in primitive to the lab's own Critic role (default max: 5 iterations)

D E M O

~4 min

Bound the loop (Option A: predicate + max_iterations, no judge)

The SAME agent and instructions run through two `should_continue` predicates. `BOUNDED` checks the marker the agent was actually told to emit — the loop stops on its own, under the cap. `RUNAWAY` checks a one-word typo of that marker (`COMPLETED` vs `COMPLETE`) that can never appear verbatim — a real bug shape where `should_continue` can never return `False` on its own, so `max_iterations` is the only thing that stops it. That's this slide's quoted warning, reproduced as code: "a completion condition can fail."

Runbook: `demos/day4/module-6-demo-1-bound-the-loop.md`

Bounding a workflow-level loop

Single-agent loop (AgentLoopMiddleware)

- Single-agent loop (AgentLoopMiddleware)
- One agent, re-invoked on itself
- max_iterations is built in

Workflow-graph loop-back (Module 3's conditional edge)

- Workflow-graph loop-back (Module 3's conditional edge)
- Two different executors — e.g., Critic routing back to Planner
- No automatic cap — track your own counter in workflow state, check it in the condition function

Tool storms

- Tool storm (this workshop's term, not framework vocabulary): an agent makes many more tool calls than the task needs — redundant lookups, retried calls that already succeeded, tool-calling in a loop with no progress
- Module 5's Tool Selection evaluator catches unnecessary tool calls directly; Tool Call Accuracy catches redundancy in the calls that were made
- A workflow multiplies this risk — a tool storm inside any one agent's turn also inflates every downstream cost metric

Cost blowouts

- The framework's own token-usage telemetry (`gen_ai.client.token.usage`, `gen_ai.usage.input_tokens/output_tokens`) is your detection signal — watch it per agent, not just for the workflow as a whole
- Magentic's `max_round_count`, `max_stall_count`, and `max_reset_count` (Module 2) are a real, shipped example of a built-in cost guardrail — not every pattern has one this explicit
- For a custom graph, the mitigation is the same as for infinite loops: a bounded counter, checked before the next round starts

Quality drift

- Quality drift (this workshop's term): a golden-set pass rate that degrades over time, or after a change that looked unrelated
- Detection is Module 5's own discipline: num_repetitions for run-to-run variance, and the eval → change → re-eval loop for change-induced drift
- The earlier a drift shows up in your regression harness, the cheaper it is to fix — this is the argument for running the loop on every change, not just before a release

Guardrails, by failure mode

Failure mode	Detect with	Mitigate with
Infinite loop (one agent)	Iteration count keeps climbing	AgentLoopMiddleware/.with_judge(...), max_iterations
Infinite loop (workflow graph)	Same executor keeps re-entering	Your own counter in workflow state, checked in the condition
Tool storm	Tool Selection / Tool Call Accuracy fail	Tighter tool descriptions; narrower instructions
Cost blowout	gen_ai.client.token.usage spikes	Built-in orchestration caps (Magentic) or your own budget check
Quality drift	Falling pass rate across repetitions or changes	num_repetitions; eval → change → re-eval on every change

Takeaways

- ✓ Every autonomous loop needs an explicit bound — the framework says this outright, and means it.
- ✓ A single agent looping on itself and a workflow graph looping between executors are bounded differently — know which one you're building.
- ✓ "Tool storm" and "quality drift" are this workshop's own names for real problems; the detection tools (Tool Selection, num_repetitions, token-usage metrics) are the framework's.
- ✓ The cheapest time to catch a regression is the next change after it was introduced — that's what a regression harness is for.